

MANUAL DE USUARIO

Versión 3.0 (Examen Práctico)

FORMULARIO DE CREACIÓN MANUAL DE USUARIOS

 Formulario de Registro

FORMULARIO

Nombre :

Apellidos :

Teléfono :

DNI :

Dirección :

Email :

Ver usuarios

Limpiar

Guardar

INDICE

1. INTRODUCCIÓN

2. REQUISITOS DEL SISTEMA

3. DESCRIPCIÓN GENERAL DE LA INTERFAZ

4. REGLAS DE VALIDACIÓN DE DATOS

5. REGISTRO Y GRABACIÓN EN FICHERO TXT (BOTÓN ACEPTAR)

6. CAMPO DE “NOTAS (OPCIONAL)”

7. BOTÓN LIMPIAR

8. ESTILO Y DISEÑO NEOMÓRFICO PASTEL

9. ACCESIBILIDAD Y USABILIDAD

10. ERRORES FRECUENTES Y SOLUCIONES

11. CÓDIGO

1. INTRODUCCIÓN

Este manual explica el funcionamiento del formulario “**Creación manual de usuarios**”, desarrollado en **Java Swing** con un estilo visual **neomórfico en tonos rosa pastel y melocotón**.

El formulario permite registrar los datos personales del usuario:

Nombre, Apellidos, Teléfono, DNI, Dirección y Correo electrónico,

además de mostrar un contador de usuarios registrados y un botón opcional para ver el contenido del fichero generado.

2. REQUISITOS DEL SISTEMA

- **Java JDK 17** o superior.
- **Windows 10/11** (o equivalente).
- Ejecución desde el IDE (IntelliJ, Eclipse, etc.) o mediante:

```
java -jar NeumorphicForm.jar
```

- Resolución mínima: **1366×768** (para visualizar correctamente los efectos pastel).

3. DESCRIPCIÓN GENERAL DE LA INTERFAZ

La aplicación muestra una ventana titulada “**Formulario de Registro**”, centrada en pantalla, con un diseño **melocotón pastel y efecto 3D suave** (estilo *Soft UI*).

Elementos principales:

- **Campos de texto (obligatorios):**
 - Nombre
 - Apellidos
 - Teléfono
 - DNI
 - Dirección
 - Email
- **Botones:**
 - **Ver usuarios** → muestra el contenido del archivo un **JTextArea**.
 - **Limpiar** → borra todos los campos y restaura el formulario.
 - **Aceptar** → valida los datos y, si son correctos, **los guarda en un archivo de texto (.txt)**.
- **Diseño:**
 - Fondo principal: #FFE9E3
 - Relieve suave con sombras y luces cálidas
 - Fuente *Segoe UI*
 - Texto en color #5A4A4A (rosa oscuro contrastado)

Formulario de Registro

FORMULARIO

Nombre :

Apellidos :

Teléfono :

DNI :

Dirección :

Email :

Ver usuarios

Limpiar

Guardar

Figura 1. Pantalla principal del formulario neomórfico pastel.

4. REGLAS DE VALIDACIÓN DE DATOS

El sistema valida todos los campos **obligatorios** antes de permitir el registro.

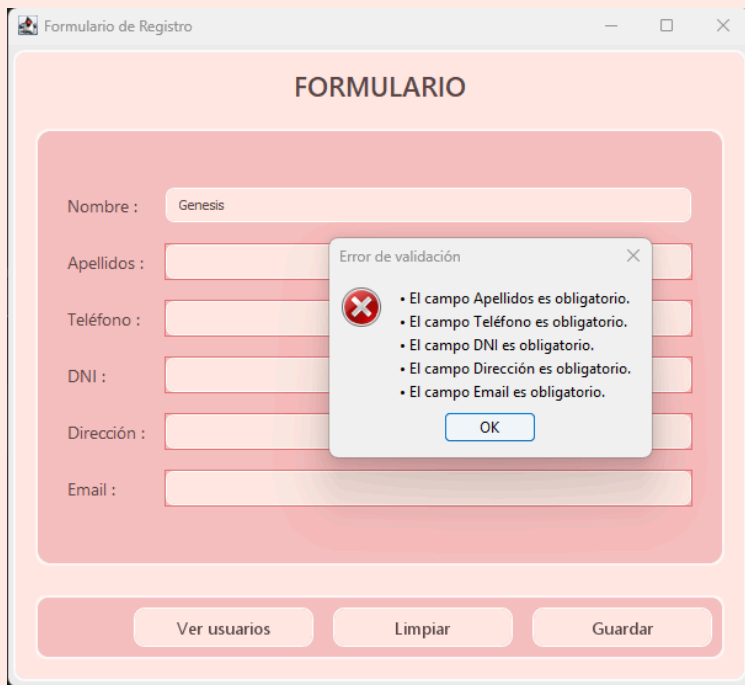
El campo “Notas” es **opcional** y no interviene en la validación.

CAMPO	REQUISITO	ACCIÓN SI FALTA CONTENIDO
Nombre	Obligatorio	Campo en rojo + mensaje de error

Apellidos	Obligatorio	Campo en rojo + mensaje de error
Teléfono	Obligatorio; 7-15 dígitos, puede empezar con “+”	Campo en rojo + mensaje de error
DNI	8 números + 1 letra (formato español)	Mensaje: “DNI debe tener 8 números y 1 letra”
Dirección	Obligatorio	Campo en rojo + mensaje de error
Email	Debe contener “@” y un “.” con al menos 2 letras después	Mensaje: “Email no válido”

Comportamiento del sistema:

- Si falta algún campo obligatorio → aparece un **cuadro de error** (JOptionPane) y los campos vacíos se marcan con **borde rojo**.
- Si todos los campos son correctos → se muestra un **mensaje de confirmación** con los datos introducidos.
- El formulario se limpia automáticamente tras aceptar.



The screenshot shows a web browser window titled "Formulario de Registro". The form has a pink background and is titled "FORMULARIO". It contains several input fields: "Nombre" (filled with "Genesis"), "Apellidos" (empty), "Teléfono" (empty), "DNI" (empty), "Dirección" (empty), and "Email" (empty). A validation error dialog box is open in the center, titled "Error de validación". It contains a red 'X' icon and a list of error messages: "El campo Apellidos es obligatorio.", "El campo Teléfono es obligatorio.", "El campo DNI es obligatorio.", "El campo Dirección es obligatorio.", and "El campo Email es obligatorio.". There is an "OK" button at the bottom of the dialog. At the bottom of the form, there are three buttons: "Ver usuarios", "Limpiar", and "Guardar".

Figura 2: Error de validación con campos vacíos.



The screenshot shows a web browser window titled "Formulario de Registro". The form has a pink background and is titled "FORMULARIO". It contains several input fields: "Nombre" (filled with "Genesis"), "Apellidos" (filled with "Vaca"), "Teléfono" (filled with "65"), "DNI" (filled with "12345678A"), "Dirección" (filled with "la que sea"), and "Email" (filled with "email@email.com"). A validation error dialog box is open in the center, titled "Error de validación". It contains a red 'X' icon and a list of error messages: "Teléfono debe tener exactamente 9 dígitos.". There is an "OK" button at the bottom of the dialog. At the bottom of the form, there are three buttons: "Ver usuarios", "Limpiar", and "Guardar".

Figura 3: Error de formato del teléfono.

The image shows a web browser window titled "Formulario de Registro". Inside, there is a form titled "FORMULARIO" with several input fields. The fields are labeled "Nombre :", "Apellidos :", "Teléfono :", "DNI :", "Dirección :", and "Email :". The values entered are "Genesis", "Vaca", "123456789", "a2s522552", "la que sea", and "email incorrecto" respectively. A modal dialog box titled "Error de validación" is overlaid on the form. It contains a red 'X' icon and two bullet points: "• DNI debe tener 8 números y 1 letra (ej.: 12345678A)." and "• Email no válido (debe contener '@' y un dominio válido).". There is an "OK" button at the bottom of the dialog. At the bottom of the form, there are three buttons: "Ver usuarios", "Limpiar", and "Guardar".

Figura 4: Error de formato DNI / email.

5. REGISTRO Y GRABACIÓN EN FICHERO TXT (BOTÓN ACEPTAR)

Cuando los datos introducidos por el usuario son **válidos**, el formulario permite almacenarlos de forma permanente en un archivo de texto:

Flujo:

1. El usuario pulsa **Grabar** (o presiona **Enter**, ya que el botón está definido como botón por defecto).
2. La aplicación **valida** todos los campos obligatorios:
 - Nombre
 - Apellidos
 - Teléfono (7–15 dígitos, opcional “+”)
 - DNI 8 dígitos + 1 Letra
 - Dirección
 - Email
3. Si hay algún error, se muestra un **mensaje de validación** y los campos problemáticos se marcan en **rojo**.
4. Si todos los datos son correctos, la aplicación **abre/crea** el archivo `usuarios_registrados.txt` (en la misma carpeta donde se ejecuta el programa) y **añade** al final del archivo la información en formato legible:

Nombre: Laura

Apellidos: Fernández López

Teléfono: 612345678

DNI: 12345678A

Dirección: Calle Mayor, 45, Madrid

Email: laura.fernandez@email.com

5. Tras guardar, se muestra un cuadro de diálogo de **éxito** indicando la ruta/nombre del fichero.



Figura 5. Registro exitoso en usuarios_registrados.txt.

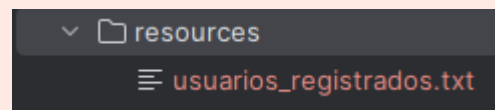


Figura 6. usuarios_registrados.txt generado.

```
Notas: Notas de Juanes
-----
Nombre: Laura
Apellidos: Fernández López
Teléfono: 612345678
DNI: 12345678A
Dirección: Calle Mayor, 45, Madrid
Email: laura.fernandez@email.com
-----
```

Figura 7. Usuario guardado en “usuarios_registrados.txt”.

6. El formulario se **limpia automáticamente** para poder registrar otro usuario.

Importante:

- El fichero se abre en **modo append**, es decir: **no borra lo anterior**, va añadiendo registros uno debajo de otro.
- Si el archivo no se puede crear (por permisos, ruta, etc.), se muestra un mensaje de **error al guardar**.

6. BOTÓN “VER USUARIOS”

Este botón abre una ventana emergente que muestra el contenido completo del archivo `usuarios_registrados.txt` dentro de un cuadro de texto (`JTextArea`).

Permite revisar todos los registros almacenados sin necesidad de abrir el archivo manualmente

La ventana incluye scroll vertical y formato legible de texto monoespaciado.

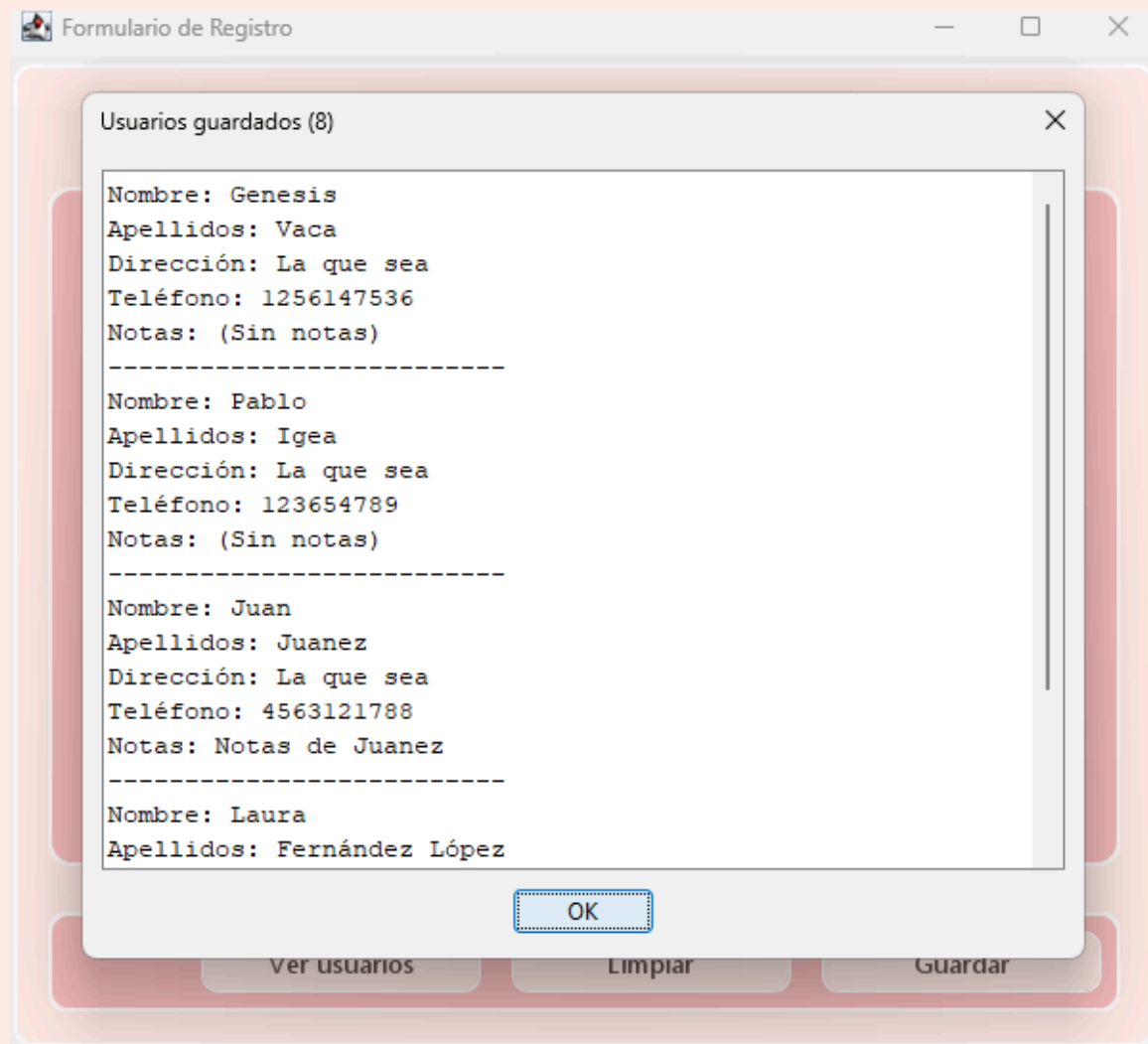


Figura 8: captura del diálogo con los registros listados.

7. BOTÓN LIMPIAR

El botón **Limpiar** borra todos los campos, incluyendo el área de notas, y restaura los bordes originales.

También devuelve el **foco** al campo “Nombre”.

Mantiene su efecto de presión neomórfico al hacer clic.

Formulario de Registro

FORMULARIO

Nombre :

Apellidos :

Teléfono :

DNI :

Dirección :

Email :

Ver usuarios

Limpiar

Guardar

Figura 9. Botón Limpiar restablece el formulario.

8. ESTILO Y DISEÑO NEOMÓRFICO PASTEL

Paleta de colores

Colores base (Rosa Melocotón Pastel)

ELEMENTO	COLOR	DESCRIPCIÓN
BASE	#FFE9E3	Fondo principal (melocotón claro)
ACCENT	#F9BEBE	Paneles y botones suaves
TEXT_DARK	#5A4A4A	Texto principal (rosa oscuro)

SHADOW_DARK	#E0B3B3	Sombra inferior del relieve
SHADOW_LIGHT	#FFFFFF	Luz superior cálida

OTRAS OPCIONES DE COLORES EN EL CÓDIGO:

Colores base (Rosa Pastel)

ELEMENTO	COLOR	DESCRIPCIÓN
BASE	#FAE8E8	Fondo principal (rosa empolvado suave)
ACCENT	#F3C6C6	Acento suave para botones o sombras interiores
TEXT_DARK	#5A4A4A	Texto gris rosado oscuro (contraste)
SHADOW_DARK	#E2BABA	Sombra inferior (tono rosado más profundo)
SHADOW_LIGHT	#FFFFFF	Luz superior cálida

Formulario de Registro

FORMULARIO

Nombre :

Apellidos :

Teléfono :

DNI :

Dirección :

Email :

Ver usuarios

Limpiar

Guardar

Figura 10. Efecto neomórfico Rosa Pastel.

Colores base (Rosa Pastel)

ELEMENTO	COLOR	DESCRIPCIÓN
BASE	#EAF0F5	Fondo principal (celeste suave)
ACCENT	#BBD4E7	Acento suave para botones o sombras interiores
TEXT_DARK	#364355	Texto gris azulado oscuro (contraste)
SHADOW_DARK	#B4C3D0	Sombra inferior (tono azulado más profundo)
SHADOW_LIGHT	#FFFFFF	Luz superior cálida

The image shows a web browser window titled 'Formulario de Registro'. The interface has a pastel blue color scheme with rounded rectangular elements. At the top, the word 'FORMULARIO' is centered in a bold, dark blue font. Below it, there are six input fields, each preceded by a label: 'Nombre :', 'Apellidos :', 'Teléfono :', 'DNI :', 'Dirección :', and 'Email :'. The 'Nombre' field contains a single character 'l'. At the bottom of the form, there are three buttons: 'Ver usuarios', 'Limpiar', and 'Guardar', all with rounded corners and a subtle gradient.

Figura 11. Efecto neomórfico Azul Pastel.

9. ACCESIBILIDAD Y USABILIDAD

- **Tecla Enter** ejecuta el botón **Aceptar**.
- **Tooltips** informativos al pasar el ratón sobre cada campo.
- **Cursor interactivo** en los botones (mano al pasar).

- Mensajes claros y centrados.
- Ventana adaptable con todos los elementos centrados.

10. ERRORES FRECUENTES Y SOLUCIONES

SITUACIÓN	CAUSA	SOLUCIÓN
Campos marcados en rojo	Campos vacíos o incorrectos	Completa todos los datos y vuelve a pulsar Aceptar
Error en teléfono	Número corto o con letras	Introduce un número válido (ej. +34911122334)
No se abre la ventana	Java no instalado	Instalar JDK y ejecutar desde el IDE
Colores apagados	Modo oscuro activo	Cambiar al modo claro del sistema
“Error al guardar los datos...”	No hay permisos o la ruta no existe	Ejecutar desde una carpeta con permisos de escritura o cambiar la ruta del <code>.txt</code> en el código
Email no válido	Falta “@” o “.”	Escribir una dirección completa (ejemplo: <code>juan@gmail.com</code>)
DNI incorrecto	No cumple formato 8 números + 1 letra	Revisar formato (ejemplo: <code>12345678A</code>)

11. CÓDIGO

```
package edu.thepower.desarrollointerfaces.examen;

/**
 * EXAMEN PRÁCTICO - Formulario Swing
 * - Campos: Nombre, Apellidos, Teléfono, DNI, Dirección, Email
 * - Botones: Guardar (valida y guarda en TXT) y Limpiar
 * - Validaciones extra: Teléfono (9 dígitos), DNI (8 números + 1
 * letra), Email con @ y '.'
 * - Guarda en usuarios.txt (formato legible) y muestra
 * confirmación
 * - Contador de usuarios guardados
 * - (Opcional) Botón "Ver usuarios" para mostrar el contenido del
 * TXT
 */

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.io.*;

public class ExamenNeumorphicForm extends JFrame {
```

```

/* // ----- Colores (neumórfico pastel) -----

private static final Color BASE = new Color(0xFFE9E3);

private static final Color ACCENT = new Color(0xF9BEBE);

private static final Color TEXT_DARK = new Color(0x5A4A4A);

private static final Color SHADOW_DARK = new
Color(0xE0B3B3);

private static final Color SHADOW_LIGHT = new
Color(0xFFFFFFFF);

/**

 * // Colores base (Azul)

 * private static final Color BASE = new
Color(0xEAF0F5); // fondo pastel

 * private static final Color ACCENT = new
Color(0xBBD4E7); // acento suave

 * private static final Color TEXT_DARK = new
Color(0x364355); // texto

 * private static final Color SHADOW_DARK = new
Color(0xB4C3D0); // sombra

 * private static final Color SHADOW_LIGHT = new
Color(0xFFFFFFFF); // luz
*/

// Colores base (rosa pastel)

private static final Color BASE = new Color(0xFAE8E8);
// Fondo principal (rosa empolvado claro)

private static final Color ACCENT = new Color(0xF3C6C6);
// Acento suave para botones o sombras interiores

private static final Color TEXT_DARK = new Color(0x5A4A4A);
// Texto gris rosado oscuro (contraste)

private static final Color SHADOW_DARK = new
Color(0xE2BABA); // Sombra inferior (tono rosado más profundo)

```

```
private static final Color SHADOW_LIGHT = new
Color(0xFFFFFF); // Luz superior (blanco suave)

// ----- Campos -----

private RoundedField txtNombre;

private RoundedField txtApellidos;

private RoundedField txtTelefono;

private RoundedField txtDni;

private RoundedField txtDireccion;

private RoundedField txtEmail;

// ----- Botones -----

private NeoButton btnGuardar;

private NeoButton btnLimpiar;

private NeoButton btnVerUsuarios;

// ----- Archivos -----

private static final String ARCHIVO =
"C:\\Users\\AlumnoAfternoon\\IdeaProjects\\Neomorphic-Form\\res
ources\\usuarios_registrados.txt";

private static final File FILETXT = new File(ARCHIVO);

// ----- Bordes -----

private final javax.swing.border.Border DEFAULT_BORDER = new
EmptyBorder(8, 12, 8, 12);

public ExamenNeomorphicForm() {
```

```

        setTitle("Formulario de Registro");

        setDefaultCloseOperation(EXIT_ON_CLOSE);

        setSize(620, 560);

        setLocationRelativeTo(null);

        try {
UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName
()); } catch (Exception ignored) {}

        UIManager.put("Label.font", new Font("Segoe UI",
Font.PLAIN, 14));

        NeoPanel root = new NeoPanel(BASE, 18);

        root.setLayout(new BorderLayout(0, 16));

        root.setBorder(new EmptyBorder(18, 18, 18, 18));

        setContentPane(root);

        JLabel title = new JLabel("FORMULARIO",
SwingConstants.CENTER);

        title.setForeground(TEXT_DARK);

        title.setFont(new Font("Segoe UI Semibold", Font.BOLD,
22));

        root.add(title, BorderLayout.NORTH);

        NeoPanel form = new NeoPanel(ACCENT, 22);

        form.setLayout(new GridBagLayout());

        form.setBorder(new EmptyBorder(20, 22, 20, 22));

        root.add(form, BorderLayout.CENTER);

        GridBagConstraints c = new GridBagConstraints();

```

```
c.insets = new Insets(8, 8, 8, 8);

c.anchor = GridBagConstraints.WEST;

c.fill = GridBagConstraints.HORIZONTAL;

c.weightx = 1;

int row = 0;

txtNombre      = addRow(form, c, row++, "Nombre :");
txtApellidos   = addRow(form, c, row++, "Apellidos :");
txtTelefono    = addRow(form, c, row++, "Teléfono :");
txtDni         = addRow(form, c, row++, "DNI :");
txtDireccion   = addRow(form, c, row++, "Dirección :");
txtEmail       = addRow(form, c, row++, "Email :");

NeoPanel actions = new NeoPanel(ACCENT, 18);
actions.setLayout(new FlowLayout(FlowLayout.RIGHT, 12,
12));

btnVerUsuarios = new NeoButton("Ver usuarios"); //
opcional

btnLimpiar = new NeoButton("Limpiar");
btnGuardar = new NeoButton("Guardar");

actions.add(btnVerUsuarios);

actions.add(btnLimpiar);

actions.add(btnGuardar);

root.add(actions, BorderLayout.SOUTH);

getRootPane().setDefaultButton(btnGuardar);
```

```

        // Acciones

        btnGuardar.addActionListener(this::onGuardar);

        btnLimpiar.addActionListener(e -> limpiar());

        btnVerUsuarios.addActionListener(e ->
mostrarUsuarios());

        // Estética campos

        for (RoundedField f : new RoundedField[]{txtNombre,
txtApellidos, txtTelefono, txtDni, txtDireccion, txtEmail}) {

            f.setForeground(TEXT_DARK);

            f.setBackground(BASE);

            f.setBorder(DEFAULT_BORDER);

        }

    }

    private RoundedField addRow(JPanel form, GridBagConstraints
c, int row, String label) {

        c.gridx = 0; c.gridy = row; c.weightx = 0;

        JLabel l = new JLabel(label);

        l.setForeground(TEXT_DARK);

        form.add(l, c);

        c.gridx = 1; c.gridy = row; c.weightx = 1;

        RoundedField field = new RoundedField(14, BASE);

        form.add(field, c);

        return field;

    }

```

```
// ----- Lógica Guardar -----

private void onGuardar(ActionEvent e) {

    resetBorders();

    String nombre      = txtNombre.getText().trim();
    String apellidos   = txtApellidos.getText().trim();
    String telefono    = txtTelefono.getText().trim();
    String dni         = txtDni.getText().trim();
    String direccion   = txtDireccion.getText().trim();
    String email       = txtEmail.getText().trim();

    StringBuilder errs = new StringBuilder();

    // Requeridos

    if (nombre.isEmpty()) { errs.append("• El campo  
Nombre es obligatorio.\n"); markError(txtNombre); }

    if (apellidos.isEmpty()) { errs.append("• El campo  
Apellidos es obligatorio.\n"); markError(txtApellidos); }

    if (telefono.isEmpty()) { errs.append("• El campo  
Teléfono es obligatorio.\n"); markError(txtTelefono); }

    if (dni.isEmpty()) { errs.append("• El campo DNI  
es obligatorio.\n"); markError(txtDni); }

    if (direccion.isEmpty()) { errs.append("• El campo  
Dirección es obligatorio.\n"); markError(txtDireccion); }

    if (email.isEmpty()) { errs.append("• El campo Email  
es obligatorio.\n"); markError(txtEmail); }

    // Teléfono 9 dígitos
```



```
        if (!telefono.isEmpty() &&
!telefono.matches("^\\d{9}$")) {

            errs.append("• Teléfono debe tener exactamente 9
dígitos.\n");

            markError(txtTelefono);

        }

        // DNI español 8 números + 1 letra (formato)

        if (!dni.isEmpty() && !dni.matches("^\\d{8}[A-Za-z]$"))
        {

            errs.append("• DNI debe tener 8 números y 1 letra
(ej.: 12345678A).\n");

            markError(txtDni);

        }

        // Email con @ y un punto con al menos 2 letras al final

        if (!email.isEmpty() &&
!email.matches("^^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]{2,}$")) {

            errs.append("• Email no válido (debe contener '@' y
un dominio válido).\n");

            markError(txtEmail);

        }

        if (errs.length() > 0) {

            JOptionPane.showMessageDialog(this, errs.toString(),
"Error de validación", JOptionPane.ERROR_MESSAGE);

            return;

        }
    }
```

```

        // Guardado en usuarios.txt (formato legible del
        enunciado)

        try (FileWriter writer = new FileWriter(FILETEXT, true)) {

            writer.write("Nombre: " + nombre + "\n");
            writer.write("Apellidos: " + apellidos + "\n");
            writer.write("Teléfono: " + telefono + "\n");
            writer.write("DNI: " + dni + "\n");
            writer.write("Dirección: " + direccion + "\n");
            writer.write("Email: " + email + "\n");
            writer.write("-----\n");

        } catch (IOException ex) {

            JOptionPane.showMessageDialog(this, "Error al
guardar:\n" + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);

            return;

        }

        // Contador de usuarios guardados (contamos líneas
        'Nombre:' o separadores)

        int total = contarUsuarios(FILETEXT);

        JOptionPane.showMessageDialog(

            this,

            "Usuario guardado correctamente en " + ARCHIVO +
            "'.\n" +

                "Usuarios almacenados: " + total,

            "Confirmación",

            JOptionPane.INFORMATION_MESSAGE

        );

```

```
        limpiar();
    }

    private int contarUsuarios(File file) {
        if (!file.exists()) return 0;

        int count = 0;

        try (BufferedReader br = new BufferedReader(new
        FileReader(file))) {
            String line;

            // Contar por separador o por 'Nombre:'

            while ((line = br.readLine()) != null) {
                if (line.startsWith("Nombre:") ||
                line.startsWith("-----")) count++;
            }

        } catch (IOException ignored) {}

        // si contamos separadores y 'Nombre:' a la vez pueden
        duplicarse; nos quedamos con el mayor

        return count / 2 == 0 ? count : Math.max(count / 2,
        count);
    }

    // ----- Opcional: Ver usuarios en JTextArea -----

    private void mostrarUsuarios() {
        if (!FILETXT.exists()) {
            JOptionPane.showMessageDialog(this, "Aún no hay
            usuarios guardados.", "Información",
            JOptionPane.INFORMATION_MESSAGE);

            return;
        }
    }
}
```

```

    }

    StringBuilder sb = new StringBuilder();

    try (BufferedReader br = new BufferedReader(new
FileReader(FILETXT))) {

        String line;

        while ((line = br.readLine()) != null)
sb.append(line).append("\n");

    } catch (IOException ex) {

        JOptionPane.showMessageDialog(this, "Error leyendo el
archivo:\n" + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);

        return;

    }

    JTextArea area = new JTextArea(sb.toString(), 20, 60);
area.setEditable(false);

area.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));

JScrollPane sp = new JScrollPane(area);

JOptionPane.showMessageDialog(this, sp, "Usuarios
guardados (" + contarUsuarios(FILETXT) + ")",
JOptionPane.PLAIN_MESSAGE);

}

// ----- Utilidades UI -----

private void limpiar() {

    txtNombre.setText("");

    txtApellidos.setText("");

    txtTelefono.setText("");

```

```

        txtDni.setText("");

        txtDireccion.setText("");

        txtEmail.setText("");

        resetBorders();

        txtNombre.requestFocus();
    }

    private void markError(JTextField f) { f.setBorder(new
    LineBorder(new Color(0xE57373), 1)); }

    private void resetBorders() {

        txtNombre.setBorder(DEFAULT_BORDER);

        txtApellidos.setBorder(DEFAULT_BORDER);

        txtTelefono.setBorder(DEFAULT_BORDER);

        txtDni.setBorder(DEFAULT_BORDER);

        txtDireccion.setBorder(DEFAULT_BORDER);

        txtEmail.setBorder(DEFAULT_BORDER);

    }

    // ----- Componentes Neumórficos -----

    static class NeoPanel extends JPanel {

        private final Color base; private final int radius;

        NeoPanel(Color base, int radius) { this.base = base;
        this.radius = radius; setOpaque(false); }

        @Override protected void paintComponent(Graphics g) {

            Graphics2D g2 = (Graphics2D) g.create();

            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

```

```

        int w = getWidth(), h = getHeight();

        g2.setColor(new Color(255, 255, 255, 180));
        g2.fillRoundRect(4, 4, w - 8, h - 8, radius, radius);

        g2.setColor(new Color(180, 190, 200, 120));
        g2.fillRoundRect(6, 6, w - 12, h - 12, radius, radius);

        g2.setColor(base); g2.fillRoundRect(6, 6, w - 12, h
- 12, radius, radius);

        g2.dispose();

        super.paintComponent(g);
    }

}

static class NeoButton extends JButton {

    NeoButton(String text) {

        super(text);

        setFocusPainted(false);

        setBorderPainted(false);

        setContentAreaFilled(false);

        setForeground(TEXT_DARK);

        setFont(new Font("Segoe UI", Font.PLAIN, 14));

        setPreferredSize(new Dimension(150, 36));

setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

    }

    @Override protected void paintComponent(Graphics g) {

        Graphics2D g2 = (Graphics2D) g.create();

        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        int w = getWidth(), h = getHeight(), r = 18;

```

```

        boolean pressed = getModel().isArmed() &&
getModel().isPressed();

        Color top = pressed ? new Color(0xE0B3B3) :
SHADOW_LIGHT;

        Color bottom = pressed ? SHADOW_LIGHT : SHADOW_DARK;

        g2.setColor(top);    g2.fillRoundRect(2, 2, w - 4, h
- 4, r, r);

        g2.setColor(bottom); g2.fillRoundRect(3, 3, w - 6, h
- 6, r, r);

        g2.setColor(BASE);    g2.fillRoundRect(3, 3, w - 6, h
- 6, r, r);

        g2.setColor(TEXT_DARK);

        FontMetrics fm = g2.getFontMetrics();

        int tx = (w - fm.stringWidth(getText())) / 2;

        int ty = (h - fm.getHeight()) / 2 + fm.getAscent();

        g2.drawString(getText(), tx, ty);

        g2.dispose();

    }

}

static class RoundedField extends JTextField {

    private final int arc; private final Color base;

    RoundedField(int arc, Color base) { this.arc = arc;
this.base = base; setOpaque(false); setBorder(new
EmptyBorder(8, 12, 8, 12)); }

    @Override protected void paintComponent(Graphics g) {

        Graphics2D g2 = (Graphics2D) g.create();

        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        int w = getWidth(), h = getHeight();

```

```
        g2.setColor(SHADOW_LIGHT); g2.fillRoundRect(1, 1, w -
2, h - 2, arc, arc);

        g2.setColor(SHADOW_DARK); g2.fillRoundRect(2, 2, w -
4, h - 4, arc, arc);

        g2.setColor(base); g2.fillRoundRect(2, 2, w
- 4, h - 4, arc, arc);

        g2.dispose();

        super.paintComponent(g);

    }

}

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> new
ExamenNeumorphicForm().setVisible(true));

}

}
```